

Peter Sestoft, Niels Hallenberg

Programming Language Concepts

Third Edition (DRAFT)

July 16, 2026

Springer

Contents

1	Introduction	1
1.1	Files for This Chapter	1
1.2	Meta Language and Object Language	1
1.3	A Simple Language of Expressions	2
1.4	Syntax and Semantics	4
1.5	Representing Expressions by Objects	5
1.6	The History of Programming Languages	7
1.7	Exercises	9
	References	12
2	Interpreters and Compilers	13
2.1	Files for This Chapter	13
2.2	Interpreters and Compilers	13
2.3	Scope and Bound and Free Variables	14
2.4	Integer Addresses Instead of Names	19
2.5	Stack Machines for Expression Evaluation	21
2.6	Postscript, a Stack-Based Language	22
2.7	Compiling Expressions to Stack Machine Code	24
2.8	Implementing an Abstract Machine in Java	25
2.9	History and Literature	25
2.10	Exercises	26
	References	29
3	From Concrete Syntax to Abstract Syntax	31
3.1	Preparatory Reading	31
3.2	Lexers, Parsers, and Generators	32
3.3	Regular Expressions in Lexer Specifications	33
3.4	Grammars in Parser Specifications	34
3.5	Working with F# Modules	35
3.6	Using <code>fslex</code> and <code>fsyacc</code>	36
3.7	Lexer and Parser Specification Examples	46

3.8	A Handwritten Recursive Descent Parser	48
3.9	JavaCC: Lexer-, Parser-, and Tree Generator	49
3.10	History and Literature	52
3.11	Exercises	53
	References	55
4	A First-Order Functional Language	57
4.1	Files for This Chapter	57
4.2	Examples and Abstract Syntax	57
4.3	Run-Time Values: Integers and Closures	59
4.4	A Simple Environment Implementation	60
4.5	Evaluating the Functional Language	60
4.6	Evaluation Rules for Micro-ML	62
4.7	Static Scope and Dynamic Scope	64
4.8	Type-Checking an Explicitly Typed Language	65
4.9	Type Rules for Monomorphic Types	68
4.10	Static Typing and Dynamic Typing	70
4.11	History and Literature	72
4.12	Exercises	73
	References	76
5	Higher-Order Functions	77
5.1	Files for This Chapter	77
5.2	Higher-Order Functions in F#	77
5.3	Higher-Order Functions in the Mainstream	78
5.4	A Higher-Order Functional Language	82
5.5	Eager and Lazy Evaluation	83
5.6	The Lambda Calculus	84
5.7	History and Literature	87
5.8	Exercises	87
	References	92
6	Polymorphic Types	93
6.1	Files for This Chapter	93
6.2	ML-Style Polymorphic Types	93
6.3	Type Rules for Polymorphic Types	97
6.4	Implementing ML Type Inference	99
6.5	Generic Types in Java and C#	104
6.6	Co-Variance and Contra-Variance	106
6.7	History and Literature	109
6.8	Exercises	109
	References	112

7	Imperative Languages	115
7.1	Files for This Chapter	115
7.2	A Naive Imperative Language	116
7.3	Environment and Store	117
7.4	Parameter Passing Mechanisms	119
7.5	The C Programming Language	120
7.6	The Micro-C Language	123
7.7	Notes on Strachey's <i>Fundamental Concepts</i>	130
7.8	History and Literature	133
7.9	Exercises	133
	References	136
8	Compiling Micro-C	137
8.1	Files for This Chapter	137
8.2	An Abstract Stack Machine	138
8.3	The Structure of the Stack at Run-Time	143
8.4	Compiling Micro-C to Abstract Machine Code	144
8.5	Compilation Schemes for Micro-C	145
8.6	Compilation of Statements	146
8.7	Compilation of Expressions	148
8.8	Compilation of Access Expressions	148
8.9	Compilation to Real Machine Code	150
8.10	History and Literature	151
8.11	Exercises	151
	References	154
9	Real-World Abstract Machines	155
9.1	Files for This Chapter	155
9.2	An Overview of Abstract Machines	155
9.3	The Java Virtual Machine (JVM)	157
9.4	The .NET Virtual Machine	162
9.5	Comparing the Bytecodes of JVM and .NET	165
9.6	Generic Types in .NET and JVM	167
9.7	Decompilers for Java and C#	170
9.8	Just-in-Time Compilation	171
9.9	Tiered JIT Compilation	173
9.10	History and Literature	175
9.11	Exercises	176
	References	177
10	Garbage Collection	179
10.1	Files for This Chapter	179
10.2	Predictable Lifetime and Stack Allocation	179
10.3	Unpredictable Lifetime and Heap Allocation	180
10.4	Allocation in a Heap	181

10.5	Garbage Collection Techniques	182
10.6	Programming with a Garbage Collector	189
10.7	Implementing a Garbage Collector in C	190
10.8	History and Literature	197
10.9	Exercises	198
	References	202
11	Continuations	205
11.1	Files for This Chapter	205
11.2	Tail-Calls and Tail-Recursive Functions	205
11.3	Continuations and Continuation-Passing Style	208
11.4	Interpreters in Continuation-Passing Style	210
11.5	The Frame Stack and Continuations	215
11.6	Exception Handling in a Stack Machine	215
11.7	Continuations and Tail Calls	217
11.8	Callcc: Call with Current Continuation	218
11.9	Continuations and Backtracking	219
11.10	History and Literature	223
11.11	Exercises	223
	References	226
12	A Locally Optimizing Compiler	229
12.1	Files for This Chapter	229
12.2	Generating Optimized Code Backwards	229
12.3	Backwards Compilation Functions	230
12.4	Other Optimizations	242
12.5	A Command Line Compiler for Micro-C	243
12.6	History and Literature	244
12.7	Exercises	244
	References	247
13	Compiling Micro-SML	249
13.1	Files for This Chapter	249
13.2	Grammar for Micro-SML	250
13.3	Type Inference for Micro-SML	258
13.4	Interpreting Micro-SML	263
13.5	Compiling Micro-SML	268
13.6	History and Literature	278
13.7	Exercises	280
	References	285
14	Typing Micro-Java	287
14.1	Files for This Chapter	287
14.2	Grammar for Micro-Java	288
14.3	Return and Reachability Analysis	294
14.4	Method Inheritance, Overriding and Overloading	296

14.5	Field Hiding and Variable Shadowing	301
14.6	Notes on Full Java	303
14.7	Type Check for Micro-Java	304
14.8	Type Check Implementation	313
14.9	History and Literature	318
14.10	Exercises	319
	References	325
15	Compiling Micro-Java	327
15.1	Files for This Chapter	327
15.2	Compilation Strategy	328
15.3	Compiling Micro-Java	339
15.4	Extensions to Abstract Machine Instruction Set	340
15.5	Compilation Schemes for Micro-Java	342
15.6	Garbage Collection for Micro-Java	346
15.7	Notes on Object-Oriented Languages	348
15.8	Exercises	349
	References	358
16	Real Machine Code: Arm64	359
16.1	Files for This Chapter	359
16.2	The Arm64 Processor Family	360
16.3	Compiling Micro-C to Arm64 Code	367
16.4	The Micro-C Arm64 Compiler	371
16.5	Compilation Schemes for Micro-C	372
16.6	Compilation of Statements	373
16.7	Compilation of Expressions	374
16.8	Compilation of Access Expressions	377
16.9	Choosing Target Registers	378
16.10	Improving the Compiler	379
16.11	History and Literature	379
16.12	Exercises	380
	References	382
A	Crash Course in F#	383
A.1	Files for This Chapter	383
A.2	Getting Started	383
A.3	Expressions, Declarations and Types	384
A.4	Program Comments	384
A.5	Pattern Matching	390
A.6	Pairs and Tuples	391
A.7	Lists	392
A.8	Records and Labels	393
A.9	Raising and Catching Exceptions	394
A.10	Datatypes	395

A.11 Type Variables and Polymorphic Functions	398
A.12 Higher-Order Functions	400
A.13 F# Mutable References	403
A.14 F# Arrays	404
A.15 Other F# Features	404
A.16 Exercises	405
References	406
Index	409